Comparativa de Estructuras de Datos Primarias y Secundarias

Esta hoja de cálculo/tabla detalla las características principales, tipos de acceso, ventajas, desventajas, complejidades de tiempo (Notación Big-O) y las operaciones más comunes de las estructuras de datos fundamentales.

Estructura de Datos	Tipo de Acceso / Principio	Descripción Corta	Operaciones Principales (Primitivas)	Operaciones Ocasionales / Funciones Auxiliares	Complejidad Temporal (Promedio)	Casos de Uso Principales
Arreglo (Array)	Acceso Directo (Índice)	Colección contigua de elementos del mismo tipo con tamaño fijo.	get(i), set(i, val)	length(), search(val), sort(), reverse(), slice()	Acceso: O(1) Búsqueda: O(n) Inserción/Borrado: O(n)	Tablas, vectores, acceso rápido por posición conocida.
Arreglo Dinámico (Dynamic Array / List)	Acceso Directo (Índice)	Arreglo que redimensiona su capacidad automáticamente al llenarse.	append(val), get(i), set(i, val), remove(i)	capacity(), resize(), indexOf(val), extend(), clear()	Acceso: O(1) Inserción al final: O(1) amortizado Inserción/Borrado	Listas generales en Python/Java, acumuladores de tamaño variable.

Estructura de Datos	Tipo de Acceso / Principio	Descripción Corta	Operaciones Principales (Primitivas)	Operaciones Ocasionales / Funciones Auxiliares	Complejidad Temporal (Promedio)	Casos de Uso Principales
					intermedio: O(n)	
Pila (Stack)	LIFO (Last In, First Out)	Estructura donde el último elemento en entrar es el primero en salir.	push(val), pop()	peek() / top(), isEmpty(), isFull(), size(), clear()	Inserción (Push): O(1) Eliminación (Pop): O(1) Búsqueda: O(n)	Pila de llamadas de funciones (Recursión), deshacer/rehacer (Ctrl+Z), evaluación de expresiones (Sintaxis).
Cola (Queue)	FIFO (First In, First Out)	Estructura donde el primer elemento en entrar es el primero en salir.	enqueue(val), dequeue()	front() / peek(), isEmpty(), isFull(), size(), rear()	Inserción (Enqueue): O(1) Eliminación (Dequeue): O(1) Búsqueda: O(n)	Colas de impresión, procesamiento de tareas en segundo plano, buffers de

Estructura de Datos	Tipo de Acceso / Principio	Descripción Corta	Operaciones Principales (Primitivas)	Operaciones Ocasionales / Funciones Auxiliares	Complejidad Temporal (Promedio)	Casos de Uso Principales
						datos.
Cola de Doble Extremo (Deque)	LIFO + FIFO (Ambos extremos)	Permite insertar y eliminar elementos tanto por el frente como por la parte trasera.	push_front(val), push_back(val), pop_front(), pop_back()	peek_front(), peek_back(), isEmpty(), rotate(), clear()	Inserción en extremos: O(1) Eliminación en extremos: O(1) Búsqueda: O(n)	Algoritmos de ventana deslizante, historial de navegación bidireccional.
Tabla Hash (Hash Table / Dictionary)	Clave - Valor (Hashing)	Estructura que asocia claves a valores usando una función hash.	insert(key, val), get(key), delete(key)	containsKey(key), keys(), values(), rehash(), hashFunction(key)	Acceso: O(1) Inserción: O(1) Eliminación: O(1) Peor caso (colisiones): O(n)	Bases de datos, indexación rápida, cachés, diccionarios, conteo de frecuencias.
Lista Enlazada Simplem	Acceso Secuencial	Nodos dispersos en	insert_head(val), delete_he	insert_after(node, val),	Acceso: O(n) Inserción	Gestión dinámica de

Estructura de Datos	Tipo de Acceso / Principio	Descripción Corta	Operaciones Principales (Primitivas)	Operaciones Ocasionales / Funciones Auxiliares	Complejidad Temporal (Promedio)	Casos de Uso Principales
ente (Singly Linked List)		memoria unidos por punteros hacia el siguiente elemento.	ad(), traverse()	search(val), reverse(), length()	al inicio: O(1) Inserción al final: O(n) o O(1) con tail	memoria, implementación de pilas y colas simples.
Lista Doblemente Enlazada (Doubly Linked List)	Acceso Secuencial Bidireccional	Nodos con enlaces hacia el elemento siguiente y el elemento anterior.	insert_head(val), insert_tail(val), delete(node)	traverse_forward(), traverse_backward(), search(val)	Acceso: O(n) Inserción/Eliminación con nodo conocido: O(1)	Sistemas de ficheros, listas de reproducción de música, caché LRU.
Conjunto Hash (HashSet)	Clave Única (Sin Duplicados)	Colección no ordenada de elementos únicos respaldada por una Tabla Hash.	add(val), remove(val), contains(val)	union(setB), intersection(setB), difference(setB), size()	Inserción: O(1) Eliminación: O(1) Búsqueda: O(1)	Eliminación de duplicados, prueba rápida de pertenencia de elementos.